Pattern Recognition Letters 14 (1993) 553-558
North-Holland

July 1993

PATREC 1074

An optimal algorithm for extracting the regions of a plane graph

X.Y. Jiang and H. Bunke

Institut für Informatik und angewandte Mathematik, Universität Bern, Länggassstrasse 51, 3012 Bern, Switzerland

Received 15 May 1992
Revised 30 September 1992

Abstract

Jiang, X.Y and H. Bunke, An optimal algorithm for extracting the regions of a plane graph, Pattern Recognition Letters 14 (1993) 553–558.

In a recent paper, Fan and Chang (1991) presented an algorithm for extracting all regions of a plane graph. It is shown in this paper that their algorithm has quadratic time and space complexity. We propose an optimal algorithm which takes $O(m \log m)$ computation time and uses $O(m)$ space, where m is the number of edges of the plane graph. The optimality of our algorithm is established by proving an $\Omega(m \log m)$ lower bound under the algebraic decision-tree model.

Keywords. Algorithms, complexity, polyhedra, surfaces, plane graphs, regions.

1. Introduction

Line drawings are a class of pictorial data where the information is conveyed by the vertices and edges. A polyhedron can be characterized by its line drawing which is generated from the intensity image taken from the polyhedron. As surfaces convey significant information, the extraction of surfaces of a line drawing is of practical interest. In a recent paper, Fan and Chang (1991) presented such an algorithm for extracting all surfaces of a line drawing.

In a more general context, a line drawing is just a plane graph, and the surfaces of the line drawing are regions of the plane graph. Thus, the general version of the surface extraction problem is: find all regions of a plane graph. An example of the problem is shown in Figure 1 where there are three regions. Clearly, the algorithm of Fan and Chang works well for the region extraction problem. As will be shown later, it has quadratic time and space complexity.

For the region extraction problem, a systolic algorithm has been proposed (Shih et al. (1989)) which takes $O(m)$ computation time and uses $O(m)$ processing elements, where m is the number of edges of the input plane graph.

In this paper, we describe an optimal sequential algorithm which takes $O(m \log m)$ computation time and uses $O(m)$ space. The optimality of our algorithm is shown by establishing an $\Omega(m \log m)$ lower bound for the region extraction problem.

The rest of this paper is organized as follows. First, we give some definitions and notations. Then we analyze the complexity of the algorithm of Fan and Chang. The new optimal algorithm is

Correspondence to: Dr. X.Y. Jiang, Institute of Informatics, University of Berne, Laenggass-Str. 51, CH-3012 Berne, Switzerland.

0167-8655/93/$06.00 © 1993 — Elsevier Science Publishers B.V. All rights reserved

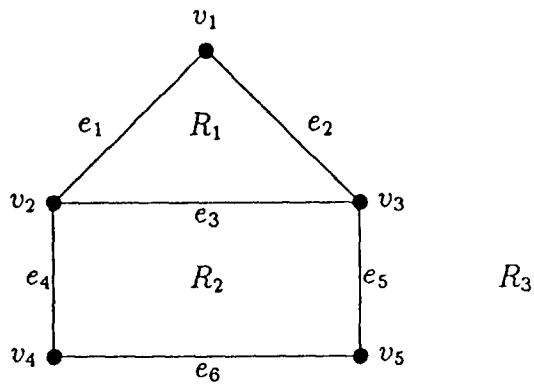

Figure 1. A plane graph and its three fundamental regions.

described in Section 4. In Section 5 we prove the optimality of our algorithm. And finally, we summarize our contribution in Section 6.

2. Definitions and notations

A graph has an *embedding* in a given space (e.g. plane) if it can be drawn in such a way that no two edges intersect except at a vertex in common. A graph is *planar* if it can be embedded in the plane. A *plane graph* is a planar graph which is embedded in the plane. It is well known that any planar graph admits of an embedding in the plane where all edges are mapped to straight line segments (Fáry (1948), Harary (1972), Stein (1951)). The input of our algorithm is a plane graph with straight line segments which is represented by a list of the undirected edges in arbitrary order. The systolic algorithm in Shih et al. (1989) implicitly assumed the same input although the authors have used the term planar graph. In the following, the term plane graph will always mean an embedding with straight line segments.

A plane graph divides the plane into *regions* (also called *windows*, *faces*, or *meshes*). A region is characterized by the set of edges forming its boundary. The plane graph in Figure 1, for instance, has three regions:

$$R_1 = ((v_1, v_2), (v_2, v_3), (v_3, v_1)),$$

$$R_2 = ((v_2, v_4), (v_4, v_5), (v_5, v_3)(v_3, v_2)),$$

$$R_3 = ((v_1, v_3), (v_3, v_5), (v_5, v_4), (v_4, v_2), (v_2, v_1)).$$

R_3 is called the *exterior region* of the plane graph which lies outside the graph and is infinite in its extent.

Given two edges $e_1 = (v_i, v_j)$ and $e_2 = (v_j, v_k)$, let us assume that we encounter no other edges incident on v_j when we rotate e_1 in clockwise direction around v_j towards e_2. Then the area comprised by e_1 and e_2 is a *wedge* between e_1 and e_2, denoted as (v_i, v_j, v_k). Two wedges (v_i, v_j, v_k) and (v_j, v_k, v_l) are said to be contiguous. It is easy to see that there are $2m$ wedges in a m-edge plane graph. Consider again the plane graph in Figure 1. Since the plane graph has six edges, there are totally twelve wedges formed. They are:

$$(v_3, v_1, v_2), \quad (v_2, v_1, v_3), \quad (v_1, v_2, v_3), \quad (v_4, v_2, v_1),$$

$$(v_3, v_2, v_4), \quad (v_2, v_3, v_1), \quad (v_5, v_3, v_2), \quad (v_1, v_3, v_5),$$

$$(v_2, v_4, v_5), \quad (v_5, v_4, v_2), \quad (v_4, v_5, v_3), \quad (v_3, v_5, v_4).$$

A region has three possible representations: a sequence of vertices, a sequence of edges, or a sequence of wedges. For example, region R_2 in Figure 1 can be represented by either of the following three sequences:

$$(v_2, v_4, v_5, v_3),$$

$$((v_2, v_4), (v_4, v_5), (v_5, v_3), (v_3, v_2)),$$

$$((v_2, v_4, v_5), (v_4, v_5, v_3), (v_5, v_3, v_2), (v_3, v_2, v_4)).$$

The three representations are equivalent and can be easily transformed into each other. Thus, either of them can be chosen for convenience.

3. The algorithm of Fan and Chang

In this section we give the algorithm of Fan and Chang in our terminology as defined in the last section. Moreover, as no complexity analysis was given in Fan and Chang (1991), we will analyze the complexity of the algorithm. In their algorithm a plane graph is represented by a $n \times n$ adjacency matrix A and a list of n entries containing the coordinates of the vertices, where n is the number of vertices of the plane graph. A_{ij} has a value 1 if the vertex v_i is connected to v_j by an edge, otherwise 0. Let m denote the number of edges of the considered plane graph. The algorithm consists of six steps:

Step 1. Choose the first vertex as the starting vertex v_s for the first surface to be extracted.

Step 2. Among the neighbors of v_s, find vertices v_p and v_q such that (v_p, v_s, v_q) builds a wedge. Relabel v_p, v_s, v_q as v_1, v_2, v_3, respectively. Record the initial region list (v_1, v_2, v_3), and remove the adjacency relationships $[v_1, v_2]$ and $[v_2, v_3]$ by setting $A_{1,2}$ and $A_{2,3}$ to 0. Initialize i to 1.

Step 3. Among the adjacent vertices of v_{i+2}, select v_{i+3} such that $(v_{i+1}, v_{i+2}, v_{i+3})$ builds a wedge. Append v_{i+3} to the region list and remove the adjacency relationship $[v_{i+2}, v_{i+3}]$ by setting $A_{i+2,i+3}$ to 0.

Step 4. If $v_{i+3} = v_1$, then the region has been extracted and go to Step 5. Otherwise increment i by 1, and go to Step 3.

Step 5. In the region list of the just extracted region $(v_1, v_2, \ldots, v_l, v_1)$, select the first vertex v_j satisfying $A_{j,j-1} = 1$ as the new starting vertex v_s and v_{j-1} as v_q, and go to Step 2. If no vertex satisfies this condition, go to Step 6.

Step 6. Check if the degrees of all vertices are zero. If so, terminate the algorithm. Otherwise, select the first vertex with nonzero degree as the new starting vertex v_s for the next region to be extracted and go to Step 2.

In the following we give a complexity analysis of the algorithm. Clearly, Step 1 can be done in constant time. If the plane graph has f regions, Step 2 will be repeated f times. Each time it needs $O(n)$ computation time for finding v_p and v_q, and constant time for other subtasks. Thus, the total time of Step 2 is $O(fn)$. For each region R_k, $k = 1, 2, \ldots, f$, Steps 3 and 4 are repeated $l_k - 2$ times, where l_k is the number of edges of R_k. Each time Step 3 needs $O(n)$ and Step 4 constant time. The total time of Steps 3 and 4 is therefore,

$$\sum_{k=1}^{f} l_k n = n \sum_{k=1}^{f} l_k = 2nm = O(mn).$$

Step 5 is repeated f times. Each time it needs $O(l_k)$ time in the worst case. Totally, Step 5 needs

$$\sum_{k=1}^{f} l_k = 2m = O(m).$$

It is difficult to determine how many times Step 6 will be repeated. However, no matter what this number is, the total time of Step 6 is always $O(n^2)$. This is because, if we find some vertex v_s with nonzero degree, we can record this vertex. The next time Step 6 is reached, we need only to search beginning at v_s. Summing up the computation cost of all six steps, the total time complexity of the algorithm is $O((f + m + n)n)$. As an adjacency matrix and a list containing the coordinates of the vertices are used for representing the plane graph, the space complexity is $O(n^2)$. Thus, both the time and space complexity are quadratic.

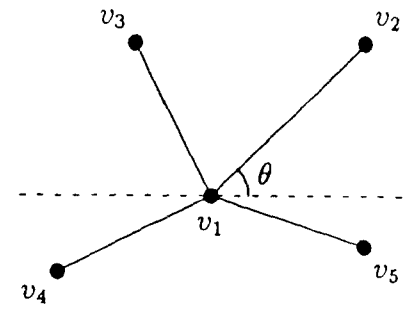

Figure 2. Four edges incident on v_1.

4. An optimal region extraction algorithm

In this section we propose an optimal algorithm which takes $O(m \log m)$ computation time and uses $O(m)$ space. The input of our algorithm is a list of undirected edges in arbitrary order. To extract regions, the overall algorithm performs two subtasks. First, we find all wedges. In a second phase, the wedges will be grouped into regions.

4.1. Phase one: finding all of the wedges

The principle of constructing wedges is simple. Consider Figure 2. There are four edges incident on v_1. Let us sort these four edges according to their angles with respect to the horizontal line passing through v_1. We get an ordered list: (v_1, v_2), (v_1, v_3), (v_1, v_4) and (v_1, v_5). Now we can consider these edges as forming a cycle by linking the last edge to the first one. Then, each pair of consecutive edges in this cycle forms a wedge.

From this consideration we can formulate the following algorithm to construct all wedges:

Step 1. Duplicate each undirected edge (v_i, v_j) to form two directed edges $\langle v_i, v_j \rangle$ (pointing from v_i to v_j) and $\langle v_j, v_i \rangle$.

Step 2. Complement each directed edge $\langle v_i, v_j \rangle$ with the angle θ of $\langle v_i, v_j \rangle$ with respect to the horizontal line passing through v_i. This enables us to compile a list with entries $(\langle v_i, v_j \rangle, \theta)$.

Step 3. Sort the list into ascending order using v_i and θ as the primary and secondary key, respectively.

Step 4. Scan the groups in the sorted list. A group consists of entries $(\langle v_i, v_j \rangle, \theta)$ of equal v_i. Within each group, combine each pair of consecutive entries $(\langle v_i, v_j \rangle, \theta_1)$ and $(\langle v_i, v_k \rangle, \theta_2)$ to build a wedge (v_k, v_i, v_j). Within each group, combine also the last and the first entry.

For the plane graph in Figure 1, the unsorted list after Steps 1 and 2 will be:

$$(\langle v_1, v_2 \rangle, \tfrac{5}{4}\pi), \quad (\langle v_2, v_1 \rangle, \tfrac{1}{4}\pi), \quad (\langle v_1, v_3 \rangle, \tfrac{7}{4}\pi),$$
$$(\langle v_3, v_1 \rangle, \tfrac{3}{4}\pi), \quad (\langle v_2, v_3 \rangle, 0), \quad (\langle v_3, v_2 \rangle, \pi),$$
$$(\langle v_2, v_4 \rangle, \tfrac{3}{2}\pi), \quad (\langle v_4, v_2 \rangle, \tfrac{1}{2}\pi), \quad (\langle v_3, v_5 \rangle, \tfrac{3}{2}\pi),$$
$$(\langle v_5, v_3 \rangle, \tfrac{1}{2}\pi), \quad (\langle v_4, v_5 \rangle, 0), \quad (\langle v_5, v_4 \rangle, \pi)$$

if we assume, for the sake of simplicity, that the input is in the order e_1, e_2, e_3, e_4, e_5 and e_6. The sorting process will produce the list:

$$(\langle v_1, v_2 \rangle, \tfrac{5}{4}\pi), \quad (\langle v_1, v_3 \rangle, \tfrac{7}{4}\pi), \quad (\langle v_2, v_3 \rangle, 0),$$
$$(\langle v_2, v_1 \rangle, \tfrac{1}{4}\pi), \quad (\langle v_2, v_4 \rangle, \tfrac{3}{2}\pi), \quad (\langle v_3, v_1 \rangle, \tfrac{3}{4}\pi),$$
$$(\langle v_3, v_2 \rangle, \pi), \quad (\langle v_3, v_5 \rangle, \tfrac{3}{2}\pi), \quad (\langle v_4, v_5 \rangle, 0),$$
$$(\langle v_4, v_2 \rangle, \tfrac{1}{2}\pi, \quad (\langle v_5, v_3 \rangle, \tfrac{1}{2}\pi), \quad (\langle v_5, v_4 \rangle, \pi).$$

Finally, Step 4 will construct the twelve wedges as listed in Section 2.

Of the four steps, Step 3 is the most expensive one in which a list of $2m$ entries has to be sorted while the other three steps have a linear time complexity, respectively. Thus, phase one of our algorithm takes $O(m \log m)$ computation time. The memory required is clearly $O(m)$.

4.2. Phase two: grouping the wedges into regions

The result of phase one is a list of wedges. In this section we show how the wedges can be grouped into regions. As seen in Section 2, a region can be represented by a sequence of wedges. It is easy to prove that a sequence of wedges $(W_1, W_2, \ldots, W_l)$ defines a region if and only if W_i and W_{i+1} are contiguous, for $i = 1, 2, \ldots, l-1$, and W_l and W_1 are also contiguous. From this observation, we can group the wedges into regions in the following way. First, we search for some initial wedge $W_1 = (v_1, v_2, v_3)$ which is still not used by previous grouping operations. Clearly, the next wedge W_2 must have the form (v_2, v_3, v_4) and can be found in the wedge list. The search will be continued until a wedge $W_l = (v_l, v_1, v_2)$ is found. Therefore, the initial grouping problem is transformed into a search problem. To facilitate the search, the wedge list will be first sorted. In the sorted wedge list the efficient binary search can be utilized.

Given the list of wedges (v_i, v_j, v_k) constructed from phase one, our grouping algorithm consists of five steps:

Step 1. Sort the wedge list using v_i and v_j as the primary and secondary key, respectively.

Step 2. Mark all wedges as unused.

Step 3. Find the next unused wedge $W_1 = (v_1, v_2, v_3)$. Record the initial region list (W_1), and initialize i to 1. Mark W_1 as used. If W_1 can not be found, the algorithm terminates and all regions have been extracted.

Step 4. Search for the wedge $W_{i+1} = (v_{i+1}, v_{i+2}, v_{i+3})$ following $W_i = (v_i, v_{i+1}, v_{i+2})$ by means of a binary search in the sorted wedge list using v_{i+1} and v_{i+2} as the primary and secondary search key, respectively. (This binary search is guaranteed to success.) Append W_{i+1} to the region list.

Step 5. If W_{i+1} and W_1 are contiguous, that is $v_{i+2} = v_1$ and $v_{i+3} = v_2$, then the region has been extracted. Go to Step 3. Otherwise increment i by 1. Go to Step 4.

For the plane graph in Figure 1, the unsorted list of wedges found in phase one is given in Section 2. The sorted wedge list will be:

$$(v_1, v_2, v_3), \quad (v_1, v_3, v_5), \quad (v_2, v_1, v_3), \quad (v_2, v_3, v_1),$$
$$(v_2, v_4, v_5), \quad (v_3, v_1, v_2), \quad (v_3, v_2, v_4), \quad (v_3, v_5, v_4),$$
$$(v_4, v_2, v_1), \quad (v_4, v_5, v_3), \quad (v_5, v_3, v_2), \quad (v_5, v_4, v_2).$$

First, (v_1, v_2, v_3) will be chosen as W_1. Next, $W_2 = (v_2, v_3, v_1)$ will be found by a binary search in the sorted wedge list using v_2 as primary key and v_3 as secondary key. The search continues to find the wedge $W_3 = (v_3, v_1, v_2)$. As W_3 and W_1 are contiguous, one region (v_1, v_2, v_3) has been detected. Then, a new cycle begins with the next unused wedge (v_1, v_3, v_5) and will find the region $(v_1, v_3, v_5, v_4, v_2)$. Finally, the third cycle will find the region (v_2, v_4, v_5, v_3).

In the formulation above, Step 1 requires $O(m \log m)$ time while step 2 is of linear complexity. The loop consisting of Steps 3, 4 and 5 takes $O(m \log m)$ computation time. Thus, the grouping algorithm has a time complexity of $O(m \log m)$. The memory requirement is clearly $O(m)$.

5. A lower bound

Summing up the costs of both phases, our region extraction algorithm has a total time complexity of $O(m \log m)$. The memory requirement is $O(m)$. In this section we prove an $\Omega(m \log m)$ lower bound for the problem of extracting regions of a plane graph with straight line segments. This implies that the algorithm given in the last section is optimal (within a constant factor).

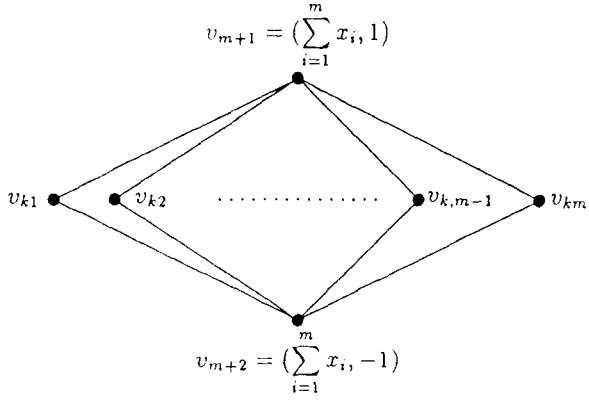

Figure 3. The first constructed plane graph.

The sorting problem is known to require $\Omega(m \log m)$ time under the algebraic decision-tree model. To prove an $\Omega(m \log m)$ lower bound for the region extraction problem, we show that any algorithm which solves this problem in time $f(m)$ can be used to solve the sorting problem in time $O(m + f(m))$.

The reduction proceeds in two steps. Given m real numbers $x_1, x_2, \ldots, x_m$, we may assume, without loss of generality, that the x_i's are all different. We construct first a plane graph with $m+2$ vertices

$$v_i = (x_i, 0), \quad i = 1, 2, \ldots, m,$$

$$v_{m+1} = \left(\sum_{i=1}^{m} x_i, 1\right), \text{ and } v_{m+2} = \left(\sum_{i=1}^{m} x_i, -1\right)$$

and $2m$ edges

$$e_i = (v_i, v_{m+1}),$$
$$e_{m+i} = (v_i, v_{m+2}), \quad i = 1, 2, \ldots, m$$

as shown in Figure 3. Obviously, this construction requires $O(m)$ time. Now we extract all regions of the constructed plane graph. They will be:

$$R_i = (v_{ki}, v_{m+2}, v_{k,i+1}, v_{m+1}), \quad i = 1, 2, \ldots, m-1$$

and

$$R_m = (v_{km}, v_{m+2}, v_{k1}, v_{m+1}).$$

Here some remarks are in order. For each extracted region R_i, $i = 1, \ldots, m-1$, we know that v_{ki} is the immediate predecessor of $v_{k,i+1}$ in the sorted list of $x_1, x_2, \ldots, x_m$. But the order in which the regions are extracted usually doesn't correspond to the correct sorting. In our algorithm description an extracted region is a sequence of wedges. As stated in Section 2, as representation of a region, a sequence of vertices and a sequence of wedges are equivalent and can be easily transformed into each other. For description convenience we use a sequence of vertices in our proof. In fact, the sequences of vertices above are in their canonical form where v_{m+2} is in the second position and v_{m+1} is in the fourth position. The canonical form of a sequence can be obtained by cyclically rotating the sequence. Finally, we can easily distinguish between R_i, $i = 1, \ldots, m-1$, and R_m. A region $(v_i = (x_i, 0), v_{m+2}, v_j = (x_j, 0), v_{m+1})$ is R_m if and only if $x_i > x_j$.

From the extracted regions, we construct in a second step another plane graph with $m+1$ vertices

$$v_i = (x_i, 0), \quad i = 1, 2, \ldots, m, \quad \text{and}$$

$$v_{m+1} = \left(\sum_{i=1}^{m} x_i, 1 \right)$$

and $m+1$ edges

$$e_i = (v_{ki}, v_{k,i+1}), \quad i = 1, 2, \ldots, m-1,$$

$$e_m = (v_{k1}, v_{m+1}) \quad \text{and} \quad e_{m+1} = (v_{km}, v_{m+1}),$$

as shown in Figure 4. Note that e_i, $i = 1, 2, \ldots, m-1$, can be constructed from the regions R_i while e_m and e_{m+1} can be formed by finding v_{k1} and v_{km} respectively from R_m. Clearly, this construction requires $O(m)$ time. Now we extract all regions of the constructed plane graph. There are two regions:

$$R_1 = (v_{k1}, v_{k2}, \ldots, v_{k,m-1}, v_{km}, v_{m+1}),$$

$$R_2 = (v_{km}, v_{k,m-1}, \ldots, v_{k2}, v_{k1}, v_{m+1}).$$

The sorted version of x_i's in ascending (descending) order can be found from R_1 (R_2).

Thus, we have the following theorem:

Theorem 1. *Sorting is linear-time transformable to the region extraction problem. Therefore, finding all regions of a plane graph with straight line segments requires $\Omega(m \log m)$ time under the algebraic decision-tree model.*

This theorem implies that our algorithm described in the last section is optimal.

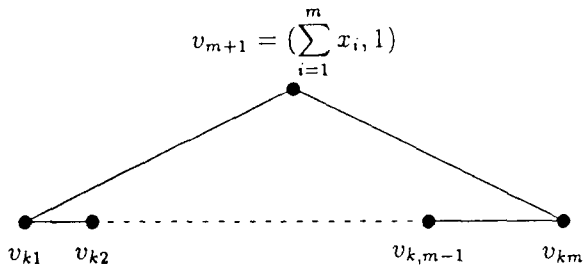

Figure 4. The second constructed plane graph.

6. Conclusion

In this paper we have described an optimal sequential algorithm for extracting all regions of a plane graph with straight line segments. The algorithm takes $O(m \log m)$ computation time and uses $O(m)$ space, where m is the number of edges of the plane graph. This improves the algorithm given in Fan and Chang (1991) which has quadratic complexity in both time and space. The optimality of our algorithm has been established by proving an $\Omega(m \log m)$ lower bound under the algebraic decision-tree model for the region extraction problem.

It is well known that the asymptotic complexity $O(f(n))$ of an algorithm is of practical interest mainly for large n values. We don't claim that our algorithm necessarily runs faster than that described in Fan and Chang (1991) if n is small, as it may be the case for the line drawing of a 3-D scene. Our concern in this paper was the region extraction problem of a plane graph in its general formulation. We have established a lower bound of the complexity of this problem and given an optimal algorithm that reaches this lower bound. However, our proposed optimal algorithm is also interesting from the application point of view as it is very easy to implement. It uses only sorting and binary search as main operations. In conclusion, it can be expected that our optimal algorithm is superior to that in Fan and Chang (1991) for large n values. But even in cases where n is small, it may still be an attractive choice, due to its ease of implementation.

References

Fan, K.-C. and C.-Y. Chang (1991). Surface extraction from line drawings of a polyhedron. *Pattern Recognition Lett.* 12, 627–633.

Fáry, I. (1948). On straight line representation of planar graphs. *Acta Sci. Math. Szeged* 11, 229–233.

Harary, F. (1972). *Graph Theory*. Addison-Wesley, Reading, MA.

Shih, Z., R.C.T. Lee and S.N. Yang (1989). A systolic algorithm for extracting regions from a planar graph. *Comp. Vision Graphics Image Process.* 47, 227–242.

Stein, S.K. (1951). Convex maps. *Proc. Amer. Math. Soc.* 2, 464–466.